SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

# Teacher guide: Day 5

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

**How to help:** first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

## Day 5: Functions and decisions

Reusable code, if and else

Yesterday you wrote instructions that run once, from top to bottom. Today you package them into **functions**: named pieces of code you can run again and again, with different numbers each time.

The page shows four show cards. Every card has grey notes like *(write watchTime)*. A helper file, **page.js**, is already written: it runs after your code and uses your functions to fill in the cards. So each function you finish makes all four cards light up at once.

You'll also learn the most important difference in today's lesson: a function can *print* a value, or *give it back*. Only one of them fills the cards.

When the cards are full, keep going with the **practice exercises** (P1 to P36): small functions of their own, from one line to a few decisions. Do them before the extra challenges.

### Step 1. Your first function

~15 min

**Teaches:** `function name() { ... }` defines a function; `name();` calls it.

#### TASK

1. Define a function called `sayHello` that logs **Hello from a function!**
2. Under it, call it twice.

#### COMMON MISTAKES

- Defining the function but never calling it (nothing appears).
- Calling it without brackets: `sayHello;` does nothing.
- Putting the calls inside the function's own `{ }`.

#### A CORRECT ANSWER

```
// app.js: your functions. page.js uses them to fill in the cards.

function sayHello() {
  console.log("Hello from a function!");
}

sayHello();
sayHello();
```

## Step 2. Parameters and arguments ~15 min

**Teaches:** Parameters are named in the definition; arguments are the values in the call.

### TASK

1. Define a function `greet` with one parameter, `name`. It logs a greeting with the name, like **Hello, Ana!**
2. Call it twice, with two different names.

### COMMON MISTAKES

- Quotes around the parameter inside the function: `"Hello, name!"`.
- Calling with no argument: `greet()` prints *Hello, undefined!*

### NOTE

If a student asks about `undefined`: it's JavaScript's word for "no value yet". It becomes important in the next two steps.

### A CORRECT ANSWER

```
// ... unchanged ...
sayHello();
sayHello();

function greet(name) {
  console.log(`Hello, ${name}!`);
}

greet("Ana");
greet("Ben");
```

## Step 3. Two parameters ~15 min

**Teaches:** Several parameters are separated by commas; arguments go in the same order.

### TASK

1. Define `watchTime` with two parameters, `episodes` and `runtime`. It logs the hours: episodes times runtime, divided by 60.
2. Call it for Friends, `watchTime(236, 30)`, and for Breaking Bad, `watchTime(62, 60)`.

Then look closely:

- The Console shows more numbers than you asked for. Where do the others come from?
- The cards say *Watch it all: undefined hours*. Why? The next step explains.

### COMMON MISTAKES

- Arguments in the wrong order (the result is the same here, but not in general).
- Using the numbers inside the function instead of the parameters: `console.log(236 * 30 / 60)`.

### A CORRECT ANSWER

```
// ... unchanged ...
greet("Ana");
greet("Ben");

function watchTime(episodes, runtime) {
  console.log(episodes * runtime / 60);
}

watchTime(236, 30);
watchTime(62, 60);
```

## Step 4. Give the answer back with return ~15 min

**Teaches:** `return` gives a value back to the caller; `console.log` only shows it to you.

### TASK

1. Change `watchTime` so that it **returns** the hours, rounded with `Math.round`, instead of logging them.
2. Your two calls now print nothing. Replace them with one line that logs the answer: `console.log(watchTime(62, 60));`.

Look at the cards: the watch times appear.

### COMMON MISTAKES

- Keeping `console.log` inside and adding `return` after it (fine, but remove the log).
- `return` on its own line with the value on the next line: JavaScript returns `undefined`.
- Writing lines after `return` and wondering why they never run.

### NOTE

This is the key idea of the day. If a student is unsure, ask: "Who needs the answer, you or page.js?"

### A CORRECT ANSWER

```
// ... unchanged ...

function watchTime(episodes, runtime) {
  return Math.round(episodes * runtime / 60);
}

console.log(watchTime(62, 60));
```

## Step 5. Decisions inside a function ~20 min

**Teaches:** Functions can use `if / else if / else` and return a different value in each block.

### TASK

Write `ratingLabel(rating)` that returns:

- **Must watch** for 9 or more
- **Good** for 8 or more
- **OK** for 6 or more
- **Skip** for anything lower

Test it with `console.log(ratingLabel(6.5));` and `console.log(ratingLabel(3));`. All our shows are rated 8 or more, so only the Console shows the last two labels.

### COMMON MISTAKES

- The conditions in the wrong order (`>= 6` first catches everything).
- `>` instead of `>=`: a rating of exactly 9 becomes Good.
- `console.log` in each block instead of `return`.

### A CORRECT ANSWER

```
// ... unchanged ...

console.log(watchTime(62, 60));

function ratingLabel(rating) {
  if (rating >= 9) {
    return "Must watch";
  } else if (rating >= 8) {
    return "Good";
  } else if (rating >= 6) {
    return "OK";
  } else {
    return "Skip";
  }
}

console.log(ratingLabel(6.5));
console.log(ratingLabel(3));
```

## Step 6. Compare two parameters ~15 min

**Teaches:** Conditions can compare parameters with each other: `if (premiered === ended)`.

### TASK

Write `yearsLabel(premiered, ended)`:

- when both years are the same, return only one year, like **2019**
- otherwise, return both with a dash between them, like **1994–2004**

The long dash `–` can be hard to type: a normal `-` is fine too.

### COMMON MISTAKES

- One `=` in the condition: it changes `premiered` and the result is always the second year.
- Returning `"premiered–ended"` in quotes instead of a template literal.

### A CORRECT ANSWER

```
// ... unchanged ...
console.log(ratingLabel(6.5));
console.log(ratingLabel(3));

function yearsLabel(premiered, ended) {
  if (premiered === ended) {
    return `${premiered}`;
  } else {
    return `${premiered}–${ended}`;
  }
}
```

## Step 7. Hours and minutes ~20 min

**Teaches:** `Math.floor()` rounds down; `%` is the remainder of a division.

### TASK

In `formatRuntime`, replace the three `TODO` comments with an `if / else if / else` that returns:

1. `"45 min"` style when `hours` is 0
2. `"1 h"` style when `rest` is 0
3. `"1 h 30 min"` style otherwise

Use template literals with `${hours}` and `${rest}`. Test it: `console.log(formatRuntime(90));`.

### COMMON MISTAKES

- Checking `minutes === 0` instead of `hours === 0`.
- Forgetting `return` in one of the blocks.

### NOTE

If `%` is confusing, use a clock: 90 minutes is 1 hour and 30 left over. `90 % 60` gives the left-over part.

### A CORRECT ANSWER

```
// ... unchanged ...
  }
}

function formatRuntime(minutes) {
  const hours = Math.floor(minutes / 60);
  const rest = minutes % 60;
  if (hours === 0) {
    return `${rest} min`;
  } else if (rest === 0) {
    return `${hours} h`;
  } else {
    return `${hours} h ${rest} min`;
  }
}

console.log(formatRuntime(90));
```

## Step 8. Functions that use functions ~15 min

**Teaches:** Functions can call other functions, and can return `true` or `false`.

### TASK

Write `isQuickWatch(episodes, runtime)`. It returns `true` when the whole show takes **10 hours or less**, and `false` otherwise.

Use your `watchTime` function inside it. Only one of the four shows is a quick watch. Can you guess which one before you look?

### COMMON MISTAKES

- Returning the strings `"true"` / `"false"` instead of booleans.
- Copying the maths from `watchTime` instead of calling it.
- `< 10` instead of `<= 10`.

### A CORRECT ANSWER

```
// ... unchanged ...

console.log(formatRuntime(90));

function isQuickWatch(episodes, runtime) {
  return watchTime(episodes, runtime) <= 10;
}
```

## Step 9. Arrow functions ~15 min

**Teaches:** Arrow functions: `const name = (a, b) => value;`

### TASK

Rewrite `watchTime` as an arrow function, on one line:

1. Write `const watchTime = (episodes, runtime) => ...` with the calculation (and `Math.round`) after the arrow.
2. Delete the old `function watchTime ...` block, from `function` to its `}`. Two functions with the same name cause an error.

The cards and the Console should look exactly the same as before.

### COMMON MISTAKES

- Keeping both versions: "Identifier 'watchTime' has already been declared".
- `{ }` around the body without `return`: the function returns `undefined`.
- Putting the arrow function below the line that calls it.

### A CORRECT ANSWER

```
// ... unchanged ...
greet("Ben");

const watchTime = (episodes, runtime) => Math.round(episodes * runtime / 60);

console.log(watchTime(62, 60));
// ... unchanged ...
```

## Step 10. A default value ~20 min

**Teaches:** Default values: `function f(a, b = "x")` uses `"x"` when `b` is left out.

### TASK

Write `plural(count, word = "episode")`:

- when `count` is 1, return the count and the word: **1 episode**
- otherwise, return the count, the word and an **s**: **236 episodes**

page.js calls `plural(episodes)` with one argument, so the default is used. Test the other word in the Console: `console.log(plural(3, "season"));` should print **3 seasons**.

### COMMON MISTAKES

- `word == "episode"` or `word: "episode"` in the brackets instead of `word = "episode"`.
- Writing `episodes` in the text instead of `${word}s`.

### NOTE

All six grey notes should now be filled. Students who finish now go on to the practice exercises P1 to P36 (the Next button leads there), and then the extra challenges.

### A CORRECT ANSWER

```
// ... unchanged ...
  return watchTime(episodes, runtime) <= 10;
}

function plural(count, word = "episode") {
  if (count === 1) {
    return `${count} ${word}`;
  } else {
    return `${count} ${word}s`;
  }
}

console.log(plural(3, "season"));
```

## Practice 1. Episodes left to watch ~5 min

**Teaches:** Practice: return a calculation made from two parameters.

### TASK

Write `episodesLeft(total, watched)`. It returns how many episodes are still left to watch.

Friends has 236 episodes, so after 10 of them, `episodesLeft(236, 10)` gives **226**.

Write your line where it says `// your code here`, then watch the ticks in the preview.

### COMMON MISTAKES

- `watched - total`, which gives negative numbers.
- `console.log` instead of `return` (the gym asks "Did you forget return?").

### NOTE

Practice exercises have their own code box on a separate page, the practice gym, so they never change the cards from the guided steps. Students can skip any of them. Every `tryIt` line becomes a row with a tick or a cross; the Check button tests more values than the rows show.

### A CORRECT ANSWER

```
// Practice 1: how many episodes are left to watch?
// episodesLeft(236, 10) should give 226.
function episodesLeft(total, watched) {
  return total - watched;
}

tryIt(episodesLeft, [236, 10], 226);
tryIt(episodesLeft, [62, 62], 0);
tryIt(episodesLeft, [5, 0], 5);
```

## Practice 2. Shout the title ~5 min

**Teaches:** Practice: string methods inside an arrow function.

### TASK

Finish the arrow function `shout(title)`. It returns the title in capital letters: `shout("Dark")` gives **"DARK"**.

Keep the curly brackets and write `return` inside, or, if you like, rewrite it on one line without `{ }` and `return`.

### COMMON MISTAKES

- Forgetting the brackets: `title.toUpperCase` gives the method itself, not the result.
- Quotes around the parameter: `"title".toUpperCase()` always gives `"TITLE"`.
- Curly brackets without `return`.

### A CORRECT ANSWER

```
// Practice 2: the title in capital letters.
// shout("Dark") should give "DARK".
const shout = (title) => {
  return title.toUpperCase();
};

tryIt(shout, ["Dark"], "DARK");
tryIt(shout, ["Breaking Bad"], "BREAKING BAD");
tryIt(shout, [""], "");
```

## Practice 3. How old is the show? ~6 min

**Teaches:** Practice: a default value in an arrow function.

### TASK

Write the arrow function `showAge(premiered, thisYear)`. It returns how many years ago the show started.

1. Give `thisYear` a default value of `2026`.
2. Return the difference between the two years.

Friends premiered in 1994, so `showAge(1994)` gives **32**, and `showAge(1994, 2004)` gives **10**.

### COMMON MISTAKES

- No default, so `showAge(1994)` gives `NaN` (2026 is missing: undefined minus 1994).
- `premiered - thisYear`, which gives negative numbers.

### NOTE

The default year 2026 is written into the task and the check. If you teach this course in a later year, students may ask why: it's only practice for default values.

### A CORRECT ANSWER

```
// Practice 3: how many years ago did the show start?
// showAge(1994) should give 32 (this year is 2026).
const showAge = (premiered, thisYear = 2026) => {
  return thisYear - premiered;
};

tryIt(showAge, [1994], 32);
tryIt(showAge, [1994, 2004], 10);
tryIt(showAge, [2019, 2019], 0);
```

## Practice 4. Does the title fit on the card? ~6 min

**Teaches:** Practice: `.length` in a comparison that returns `true` or `false`.

### TASK

A card has room for 12 characters on one line. Write `fitsOnCard(title)`: it returns `true` when the title has **12 characters or fewer**, and `false` otherwise.

"Breaking Bad" has exactly 12 characters (count the space), so it fits.

### COMMON MISTAKES

- `title.length()` with brackets: "title.length is not a function".
- `< 12` instead of `<= 12`, so "Breaking Bad" doesn't fit.
- Returning `"true"` in quotes, which is text, not a boolean.

### A CORRECT ANSWER

```
// Practice 4: does the title fit in 12 characters?
// fitsOnCard("Dark") should give true.
function fitsOnCard(title) {
  return title.length <= 12;
}

tryIt(fitsOnCard, ["Dark"], true);
tryIt(fitsOnCard, ["Breaking Bad"], true);
tryIt(fitsOnCard, ["Game of Thrones"], false);
tryIt(fitsOnCard, [""], true);
```

## Practice 5. One sentence about a show ~7 min

**Teaches:** Practice: a template literal built from four parameters.

### TASK

Write `aboutShow(name, episodes, runtime, network)`. It returns one sentence like this:

**Fleabag has 12 episodes of 30 minutes on BBC Three.**

Use all four parameters, so the same function works for every show.

### COMMON MISTAKES

- Normal quotes instead of backticks: the `${ }` parts appear as they are.
- Typing "Fleabag" into the sentence instead of `${name}`.
- A missing space around a `${ }` part, so words stick together.

### A CORRECT ANSWER

```
// Practice 5: one sentence about a show, from four parameters.
// aboutShow("Fleabag", 12, 30, "BBC Three") should give
// "Fleabag has 12 episodes of 30 minutes on BBC Three."
function aboutShow(name, episodes, runtime, network) {
  return `${name} has ${episodes} episodes of ${runtime} minutes on ${network}.`;
}

tryIt(aboutShow, ["Fleabag", 12, 30, "BBC Three"], "Fleabag has 12 episodes of 30 minutes on BBC
Three.");
tryIt(aboutShow, ["Dark", 26, 60, "Netflix"], "Dark has 26 episodes of 60 minutes on Netflix.");
```

## Practice 6. Stars out of five ~6 min

**Teaches:** Practice: `Math.round` around a calculation.

### TASK

Write the arrow function `stars(rating)`. It returns the rating divided by 2 and rounded to a whole number.

Friends is rated 8.8, so `stars(8.8)` gives **4**. Breaking Bad's 9.2 gives **5**.

### COMMON MISTAKES

- `Math.round(rating) / 2`: rounds first, divides second, and gives 4.5.
- `Math.floor` instead of `Math.round`: 9.0 becomes 4 stars.

### A CORRECT ANSWER

```
// Practice 6: a rating out of 10 as whole stars out of 5.
// stars(8.8) should give 4.
const stars = (rating) => Math.round(rating / 2);

tryIt(stars, [8.8], 4);
tryIt(stars, [9.2], 5);
tryIt(stars, [0], 0);
```

## Practice 7. How much have I watched? ~8 min

**Teaches:** Practice: percentages with `/`, `*` and `Math.round`.

### TASK

Write `percentWatched(watched, total)`. It returns the percentage of the show you've watched, rounded to a whole number.

Breaking Bad has 62 episodes, so `percentWatched(31, 62)` gives **50**. Friends has 236, so 10 of them is **4**.

### COMMON MISTAKES

- Forgetting `* 100`, so the answer is 0.5 instead of 50.
- `total / watched`, the wrong way round.
- No rounding, so 10 of 236 gives 4.237...

### A CORRECT ANSWER

```
// Practice 7: what percentage of the show have you watched?
// percentWatched(31, 62) should give 50.
function percentWatched(watched, total) {
  return Math.round(watched / total * 100);
}

tryIt(percentWatched, [31, 62], 50);
tryIt(percentWatched, [10, 236], 4);
tryIt(percentWatched, [0, 26], 0);
tryIt(percentWatched, [12, 12], 100);
```

## Practice 8. Classic or modern? ~6 min

**Teaches:** Practice: `if / else` with a boundary value.

### TASK

Write `era(year)`. It returns **"Classic"** for shows that premiered before 2000, and **"Modern"** for 2000 and later.

So Friends (1994) is Classic, and Band of Brothers (2001) is Modern. A show from exactly 2000 is Modern.

### COMMON MISTAKES

- `<= 2000`, so 2000 becomes Classic.
- `console.log("Classic")` instead of `return "Classic"`.

### A CORRECT ANSWER

```
// Practice 8: "Classic" before 2000, "Modern" from 2000 on.
// era(1994) should give "Classic".
function era(year) {
  if (year < 2000) {
    return "Classic";
  } else {
    return "Modern";
  }
}

tryIt(era, [1994], "Classic");
tryIt(era, [2008], "Modern");
tryIt(era, [2000], "Modern");
```

## Practice 9. Can I finish it tonight? ~8 min

**Teaches:** Practice: return a comparison that has a calculation on one side.

### TASK

Write `canFinishTonight(episodes, runtime, minutesFree)`. It returns `true` when all the episodes, one after another, fit into the minutes you have free, and `false` otherwise.

Chernobyl has 5 episodes of 60 minutes, so 300 free minutes are exactly enough: `canFinishTonight(5, 60, 300)` gives **true**.

### COMMON MISTAKES

- `<` instead of `<=`: 5 episodes of 60 minutes don't "fit" into 300 minutes.
- Comparing only `runtime` with `minutesFree`, forgetting the number of episodes.

### A CORRECT ANSWER

```
// Practice 9: do all the episodes fit into your free evening?
// canFinishTonight(5, 60, 300) should give true.
function canFinishTonight(episodes, runtime, minutesFree) {
  return episodes * runtime <= minutesFree;
}

tryIt(canFinishTonight, [5, 60, 300], true);
tryIt(canFinishTonight, [4, 30, 90], false);
tryIt(canFinishTonight, [0, 60, 0], true);
```

## Practice 10. Minutes as hours and minutes ~8 min

**Teaches:** Practice: `Math.floor` and `%` to split minutes.

### TASK

Write `hoursAndMinutes(minutes)`. It **always** returns both parts, like **"2 h 15 min"**, even when one of them is 0: `hoursAndMinutes(45)` gives **"0 h 45 min"**.

Watching all of The Office takes 6030 minutes. What does your function make of that?

### COMMON MISTAKES

- `Math.round(minutes / 60)`: 6030 minutes becomes 101 hours.
- No `Math.floor` at all, so the hours have decimals: "2.25 h".
- Normal quotes instead of backticks.

### A CORRECT ANSWER

```
// Practice 10: minutes as "2 h 15 min", always with both parts.
// hoursAndMinutes(135) should give "2 h 15 min".
function hoursAndMinutes(minutes) {
  const hours = Math.floor(minutes / 60);
  const rest = minutes % 60;
  return `${hours} h ${rest} min`;
}

tryIt(hoursAndMinutes, [135], "2 h 15 min");
tryIt(hoursAndMinutes, [60], "1 h 0 min");
tryIt(hoursAndMinutes, [45], "0 h 45 min");
tryIt(hoursAndMinutes, [6030], "100 h 30 min");
```

## Practice 11. Round a rating to one decimal ~8 min

**Teaches:** Practice: round to one decimal with `Math.round(x * 10) / 10`.

### TASK

A rating worked out from many votes can look like 8.7634. Write the arrow function `roundRating(rating)` that rounds it to one decimal: `roundRating(8.76)` gives **8.8**.

### COMMON MISTAKES

- `Math.round(rating) / 10` or `Math.round(rating / 10)`: the brackets in the wrong place.
- Forgetting `/ 10` at the end, so 8.76 becomes 88.
- `rating.toFixed(1)`, which gives the text "8.8".

### A CORRECT ANSWER

```
// Practice 11: round a rating to one decimal.
// roundRating(8.76) should give 8.8.
const roundRating = (rating) => Math.round(rating * 10) / 10;

tryIt(roundRating, [8.76], 8.8);
tryIt(roundRating, [9.04], 9);
tryIt(roundRating, [8.333], 8.3);
```

## Practice 12. Not started, watching or finished? ~12 min

**Teaches:** Practice: four cases with `if / else if / else`, in the right order.

### TASK

Write `progressLabel(watched, total)` with four cases:

1. **"Not started"** when `watched` is 0
2. **"Finished"** when `watched` is `total`
3. **"Last episode"** when exactly one episode is left
4. **"Watching"** otherwise

Think about the order. A one-episode show you haven't started has 0 watched *and* one episode left: it should say "Not started".

### COMMON MISTAKES

- Testing "Last episode" before "Not started", so `progressLabel(0, 1)` says "Last episode".
- One `=` instead of `===` in a condition.
- `total - 1 === watched` written as `total - 1 = watched` (an error).

### A CORRECT ANSWER

```
// Practice 12: a label for your progress through a show.
// progressLabel(0, 236) should give "Not started".
function progressLabel(watched, total) {
  if (watched === 0) {
    return "Not started";
  } else if (watched === total) {
    return "Finished";
  } else if (watched === total - 1) {
    return "Last episode";
  } else {
    return "Watching";
  }
}

tryIt(progressLabel, [0, 236], "Not started");
tryIt(progressLabel, [1, 236], "Watching");
tryIt(progressLabel, [235, 236], "Last episode");
tryIt(progressLabel, [236, 236], "Finished");
tryIt(progressLabel, [0, 1], "Not started");
```

## Practice 13. Price per episode ~10 min

**Teaches:** Practice: an early `return` for 0, and rounding to two decimals.

### TASK

Imagine a box set that costs `price`. Write `pricePerEpisode(price, episodes)`. It returns the price of one episode, rounded to two decimals.

- When `episodes` is 0, return 0.
- A 30-euro Fleabag box set (12 episodes) gives **2.5**. 20 euros for 3 episodes gives **6.67**.

### COMMON MISTAKES

- No check for 0: `pricePerEpisode(15, 0)` gives `Infinity`.
- Rounding with 10 instead of 100, so 6.67 becomes 6.7.
- `toFixed(2)`, which gives text.

### A CORRECT ANSWER

```
// Practice 13: the price of one episode, rounded to cents.
// pricePerEpisode(30, 12) should give 2.5. With 0 episodes, give 0.
function pricePerEpisode(price, episodes) {
  if (episodes === 0) {
    return 0;
  }
  return Math.round(price / episodes * 100) / 100;
}

tryIt(pricePerEpisode, [30, 12], 2.5);
tryIt(pricePerEpisode, [20, 3], 6.67);
tryIt(pricePerEpisode, [15, 0], 0);
```

## Practice 14. Shorten long titles ~10 min

**Teaches:** Practice: `.slice(0, n)`, `.length` and a default value together.

### TASK

Write `shortTitle(title, max)`:

1. Give `max` a default value of `10`.
2. When the title is **longer** than `max`, return its first `max` characters plus `"..."`.
3. Otherwise, return the title as it is.

`shortTitle("Game of Thrones")` gives **"Game of Th..."**. "The Office" has exactly 10 characters, so it stays as it is.

### COMMON MISTAKES

- `>=` instead of `>`, so "The Office" (exactly 10) gets dots.
- Writing `10` inside the function instead of `max`, so the second argument is ignored.
- No default, so `shortTitle("Game of Thrones")` keeps the whole title.

### A CORRECT ANSWER

```
// Practice 14: cut titles that are longer than max, and add "...".
// shortTitle("Game of Thrones") should give "Game of Th...".
function shortTitle(title, max = 10) {
  if (title.length > max) {
    return title.slice(0, max) + "...";
  } else {
    return title;
  }
}

tryIt(shortTitle, ["Game of Thrones"], "Game of Th...");
tryIt(shortTitle, ["Dark"], "Dark");
tryIt(shortTitle, ["The Office"], "The Office");
tryIt(shortTitle, ["Chernobyl", 5], "Chern...");
```

## Practice 15. Old enough, and enough time? ~8 min

**Teaches:** Practice: `&&` to require two conditions at once.

### TASK

Write `canWatchTonight(age, ageLimit, runtime, minutesFree)`. It returns `true` only when **both** are true:

- `age` is at least `ageLimit` (exactly the limit is fine)
- one episode ( `runtime` ) fits into `minutesFree` (exactly is fine too)

`canWatchTonight(16, 16, 60, 60)` gives **true**.

### COMMON MISTAKES

- Only one of the two comparisons, so a too-young viewer gets `true`.
- `>` instead of `>=`, so exactly the age limit isn't allowed.
- Writing `age >= ageLimit && <= minutesFree`: each side needs its own complete comparison.

### A CORRECT ANSWER

```
// Practice 15: old enough AND enough time for one episode?
// canWatchTonight(16, 16, 60, 60) should give true.
function canWatchTonight(age, ageLimit, runtime, minutesFree) {
  return age >= ageLimit && runtime <= minutesFree;
}

tryIt(canWatchTonight, [16, 16, 60, 60], true);
tryIt(canWatchTonight, [15, 16, 60, 90], false);
tryIt(canWatchTonight, [30, 18, 60, 45], false);
```

## Practice 16. How many evenings? (harder) ~12 min

**Teaches:** Practice: round up with `Math.floor`, `%` and an `if`.

### TASK

You watch `perEvening` episodes every evening. Write `eveningsNeeded(episodes, perEvening)`. It returns how many evenings it takes to watch all the episodes. A last evening with fewer episodes still counts.

Fleabag has 12 episodes: at 3 a night that's **4** evenings. Chernobyl's 5 episodes at 2 a night take **3**. And 0 episodes take **0** evenings.

### COMMON MISTAKES

- Always adding 1, so 12 episodes at 3 a night become 5 evenings.
- `Math.round(episodes / perEvening)`: 10 episodes at 3 a night is 3.33, which rounds to 3, one evening short.

### NOTE

Students who did the extra challenge *How many days to binge it?* may know `Math.ceil`. `return Math.ceil(episodes / perEvening);` is also correct and passes.

### A CORRECT ANSWER

```
// Practice 16 (harder): how many evenings to watch every episode?
// eveningsNeeded(12, 3) should give 4, eveningsNeeded(5, 2) should give 3.
function eveningsNeeded(episodes, perEvening) {
  const full = Math.floor(episodes / perEvening);
  if (episodes % perEvening === 0) {
    return full;
  } else {
    return full + 1;
  }
}

tryIt(eveningsNeeded, [12, 3], 4);
tryIt(eveningsNeeded, [5, 2], 3);
tryIt(eveningsNeeded, [62, 4], 16);
tryIt(eveningsNeeded, [0, 3], 0);
```

## Practice 17. Time left, from two helpers (harder) ~15 min

**Teaches:** Practice: a function that calls two other functions and chooses between three answers.

### TASK

Write `timeLeft(watched, total, runtime)`. Use `episodesLeft` to work out how many episodes are left, then return:

1. **"Finished!"** when 0 or fewer are left (yes, people type the wrong numbers)
2. **"Last one! 30 min to go"** when exactly one is left, with `formatRuntime(runtime)` for the time
3. **"2 episodes left: 1 h to go"** otherwise, with `formatRuntime` of the minutes for **all** the episodes left

### COMMON MISTAKES

- `formatRuntime(runtime)` in the last case, so the time is for one episode, not all of them.
- `left === 0` instead of `left <= 0`, so watching "too many" episodes gives "-8 episodes left".
- Arguments to `episodesLeft` in the wrong order: it wants `(total, watched)`.

### NOTE

The order of the arguments is the trap here: `timeLeft` gets `(watched, total, ...)`, but `episodesLeft` wants `(total, watched)`. Swapped, every show looks finished.

### A CORRECT ANSWER

```
// Practice 17 (harder): how much is left? Use the two helpers.
// timeLeft(10, 12, 30) should give "2 episodes left: 1 h to go".

// Helpers, already written (don't change them):
function episodesLeft(total, watched) {
  return total - watched;
}

function formatRuntime(minutes) {
  const hours = Math.floor(minutes / 60);
  const rest = minutes % 60;
  if (hours === 0) {
    return `${rest} min`;
  } else if (rest === 0) {
    return `${hours} h`;
  } else {
    return `${hours} h ${rest} min`;
  }
}

// Your function:
function timeLeft(watched, total, runtime) {
  const left = episodesLeft(total, watched);
  if (left <= 0) {
    return "Finished!";
  } else if (left === 1) {
    return `Last one! ${formatRuntime(runtime)} to go`;
  } else {
    return `${left} episodes left: ${formatRuntime(left * runtime)} to go`;
  }
}

tryIt(timeLeft, [10, 12, 30], "2 episodes left: 1 h to go");
tryIt(timeLeft, [200, 201, 30], "Last one! 30 min to go");
tryIt(timeLeft, [100, 201, 30], "101 episodes left: 50 h 30 min to go");
tryIt(timeLeft, [62, 62, 60], "Finished!");
```

## Practice 18. What's next? (harder) ~15 min

**Teaches:** Practice: three cases from four parameters, with calculations inside `${ }`.

### TASK

Write `nextEpisode(season, episode, perSeason, seasons)`. It gets the episode you watched last, the number of episodes per season and the number of seasons, and returns:

1. **"Season 1, episode 4"** when the season has more episodes
2. **"Season 2, episode 1"** when the season is over but there's another season
3. **"That was the finale!"** when it was the last episode of the last season

### COMMON MISTAKES

- `${episode} + 1` instead of `${episode + 1}`: the page shows "3 + 1".
- `episode <= perSeason`, so after episode 6 comes "episode 7".
- Testing `season < seasons` first, so the middle of season 1 jumps to season 2.

### A CORRECT ANSWER

```
// Practice 18 (harder): which episode comes next?
// Fleabag: 2 seasons of 6 episodes.
// nextEpisode(1, 6, 6, 2) should give "Season 2, episode 1".
function nextEpisode(season, episode, perSeason, seasons) {
  if (episode < perSeason) {
    return `Season ${season}, episode ${episode + 1}`;
  } else if (season < seasons) {
    return `Season ${season + 1}, episode 1`;
  } else {
    return "That was the finale!";
  }
}

tryIt(nextEpisode, [1, 3, 6, 2], "Season 1, episode 4");
tryIt(nextEpisode, [1, 6, 6, 2], "Season 2, episode 1");
tryIt(nextEpisode, [2, 6, 6, 2], "That was the finale!");
tryIt(nextEpisode, [1, 5, 5, 1], "That was the finale!");
```

## Practice 19. Minutes to seconds ~5 min

**Teaches:** Practice: a one-line converter with an arrow function.

### TASK

Write the arrow function `toSeconds(minutes)`. It returns the same time in seconds.

A Friends episode lasts 30 minutes, so `toSeconds(30)` gives **1800**.

### COMMON MISTAKES

- Dividing by 60 instead of multiplying.
- Curly brackets without `return`.

### A CORRECT ANSWER

```
// Practice 19: a converter from minutes to seconds.
// toSeconds(30) should give 1800.
const toSeconds = (minutes) => minutes * 60;

tryIt(toSeconds, [30], 1800);
tryIt(toSeconds, [1], 60);
tryIt(toSeconds, [0], 0);
```

## Practice 20. Hours to whole days ~5 min

**Teaches:** Practice: divide and round down for whole units.

### TASK

Write the arrow function `fullDays(hours)`. It returns how many **whole** days fit into the hours.

Watching all of Friends takes 118 hours, so `fullDays(118)` gives **4**: four full days and some hours left over.

### COMMON MISTAKES

- `Math.round`, so 23 hours counts as a whole day.
- Multiplying by 24 instead of dividing.

### A CORRECT ANSWER

```
// Practice 20: how many whole days fit into some hours?
// fullDays(118) should give 4.
const fullDays = (hours) => Math.floor(hours / 24);

tryIt(fullDays, [118], 4);
tryIt(fullDays, [24], 1);
tryIt(fullDays, [23], 0);
```

## Practice 21. Title with its year ~6 min

**Teaches:** Practice: a string builder with a check for the empty string.

### TASK

Write `titleWithYear(name, year)`. It returns the name and the year in brackets: `titleWithYear("Dark", 2017)` gives **"Dark (2017)"**.

When `name` is an empty string, use **Untitled** instead: `titleWithYear("", 2005)` gives **"Untitled (2005)"**.

### COMMON MISTAKES

- `${(year)}` or `$(year)` instead of `(${year})`.
- Forgetting the space before the bracket: "Dark(2017)".
- Checking `name === " "` (a space) instead of `""`.

### A CORRECT ANSWER

```
// Practice 21: a show's name with its year in brackets.
// titleWithYear("Dark", 2017) should give "Dark (2017)".
function titleWithYear(name, year) {
  if (name === "") {
    return `Untitled (${year})`;
  }
  return `${name} (${year})`;
}

tryIt(titleWithYear, ["Dark", 2017], "Dark (2017)");
tryIt(titleWithYear, ["The Wire", 2002], "The Wire (2002)");
tryIt(titleWithYear, ["", 2005], "Untitled (2005)");
```

## Practice 22. Which show came first? ~7 min

**Teaches:** Practice: compare two shows passed as name and number.

### TASK

Write `olderShow(nameA, yearA, nameB, yearB)`. It returns the name of the show that premiered **first**, or **"Same year"** when both started in the same year.

Friends (1994) came before The Office (2005), so `olderShow("Friends", 1994, "The Office", 2005)` gives **"Friends"**.

### COMMON MISTAKES

- `>` instead of `<`: the newer show wins.
- Returning `"nameA"` in quotes, which gives the word nameA.
- No draw case, so same-year shows return the second name.

### A CORRECT ANSWER

```
// Practice 22: which of two shows premiered first?
// olderShow("Friends", 1994, "The Office", 2005) should give "Friends".
function olderShow(nameA, yearA, nameB, yearB) {
  if (yearA < yearB) {
    return nameA;
  } else if (yearB < yearA) {
    return nameB;
  } else {
    return "Same year";
  }
}

tryIt(olderShow, ["Friends", 1994, "The Office", 2005], "Friends");
tryIt(olderShow, ["Dark", 2017, "The Wire", 2002], "The Wire");
tryIt(olderShow, ["Fleabag", 2016, "The Crown", 2016], "Same year");
```

## Practice 23. Who picks tonight? Even or odd ~6 min

**Teaches:** Practice: even or odd with `% 2`.

### TASK

You and a friend take turns choosing what to watch. Write `whoPicks(evening)`:

- odd evenings (1, 3, 5 ...): return **"You"**
- even evenings (2, 4, 6 ...): return **"Your friend"**

### COMMON MISTAKES

- `evening / 2 === 0`, which is only true for 0.
- The two labels in the wrong blocks.

### A CORRECT ANSWER

```
// Practice 23: odd evenings you pick, even evenings your friend picks.
// whoPicks(1) should give "You".
function whoPicks(evening) {
  if (evening % 2 === 0) {
    return "Your friend";
  } else {
    return "You";
  }
}

tryIt(whoPicks, [1], "You");
tryIt(whoPicks, [2], "Your friend");
tryIt(whoPicks, [61], "You");
```

## Practice 24. Is the rating valid? ~6 min

**Teaches:** Practice: check a range with `>=`, `<=` and `&&`.

### TASK

Write the arrow function `isValidRating(rating)`. It returns `true` when the rating is from 0 to 10 (both included), and `false` otherwise.

### COMMON MISTAKES

- `0 <= rating <= 10`: JavaScript reads it differently and says `true` for 15.
- `>` and `<`, so 0 and 10 are rejected.

### A CORRECT ANSWER

```
// Practice 24: is the rating from 0 to 10?
// isValidRating(8.8) should give true, isValidRating(11) false.
const isValidRating = (rating) => rating >= 0 && rating <= 10;

tryIt(isValidRating, [8.8], true);
tryIt(isValidRating, [11], false);
tryIt(isValidRating, [-1], false);
tryIt(isValidRating, [10], true);
```

## Practice 25. Do the years make sense? ~8 min

**Teaches:** Practice: validation that names the problem.

### TASK

Write `checkYears(premiered, ended)`. It returns:

1. **"Not out yet"** when `premiered` is after 2026
2. **"Ended before it started"** when `ended` is before `premiered`
3. **"OK"** otherwise

Chernobyl premiered and ended in 2019: that's **"OK"**.

### COMMON MISTAKES

- `ended <= premiered`, so a one-year show like Chernobyl is rejected.
- `>= 2026`, so a show from this year is "Not out yet".

### A CORRECT ANSWER

```
// Practice 25: do the two years make sense?
// checkYears(2013, 2008) should give "Ended before it started".
function checkYears(premiered, ended) {
  if (premiered > 2026) {
    return "Not out yet";
  } else if (ended < premiered) {
    return "Ended before it started";
  } else {
    return "OK";
  }
}

tryIt(checkYears, [1994, 2004], "OK");
tryIt(checkYears, [2013, 2008], "Ended before it started");
tryIt(checkYears, [2030, 2035], "Not out yet");
tryIt(checkYears, [2019, 2019], "OK");
```

## Practice 26. Price tiers for screens ~8 min

**Teaches:** Practice: a price table as a chain of cases.

### TASK

Imagine a streaming plan whose price depends on how many screens can watch at once. Write `planPrice(screens)`:

- 0 screens: **0**
- 1 screen: **8**
- 2 screens: **12**
- 3 or more screens: **16**

### COMMON MISTAKES

- `else if (screens === 3)` for the last tier, so 4 screens gives `undefined`.
- Prices in quotes ( `"8"` ), which are text, not numbers.

### A CORRECT ANSWER

```
// Practice 26: the price of a plan, by number of screens.
// planPrice(2) should give 12.
function planPrice(screens) {
  if (screens === 0) {
    return 0;
  } else if (screens === 1) {
    return 8;
  } else if (screens === 2) {
    return 12;
  } else {
    return 16;
  }
}

tryIt(planPrice, [0], 0);
tryIt(planPrice, [1], 8);
tryIt(planPrice, [2], 12);
tryIt(planPrice, [5], 16);
```

## Practice 27. Age groups ~6 min

**Teaches:** Practice: sort a number into groups, smallest first.

### TASK

ShowShelf sorts viewers into groups. Write `ageGroup(age)`:

- under 7: **"Little ones"**
- 7 to 12: **"Kids"**
- 13 to 17: **"Teens"**
- 18 and older: **"Adults"**

### COMMON MISTAKES

- `<= 13` for Kids, so a 13-year-old is a Kid.
- Testing `age < 18` first, so everyone under 18 is a Teen.

### A CORRECT ANSWER

```
// Practice 27: sort a viewer's age into a group.
// ageGroup(13) should give "Teens".
function ageGroup(age) {
  if (age < 7) {
    return "Little ones";
  } else if (age < 13) {
    return "Kids";
  } else if (age < 18) {
    return "Teens";
  } else {
    return "Adults";
  }
}

tryIt(ageGroup, [4], "Little ones");
tryIt(ageGroup, [7], "Kids");
tryIt(ageGroup, [13], "Teens");
tryIt(ageGroup, [18], "Adults");
```

## Practice 28. A show for your mood ~8 min

**Teaches:** Practice: compare text with `===` after `.toLowerCase()`.

### TASK

Write `recommend(mood)`, which picks a show from our table by genre:

- **"happy"**: a comedy, **"Friends"**
- **"curious"**: a mystery, **"Dark"**
- **"brave"**: an adventure, **"Avatar: The Last Airbender"**
- anything else: **"The Crown"**

It must work whatever capitals are used: `recommend("Happy")` and `recommend("HAPPY")` both give "Friends".

### COMMON MISTAKES

- Comparing `mood === "happy"` without `.toLowerCase()`, so "Happy" gives The Crown.
- Comparing with `"Happy"` (capital H) after making the text small.
- `mood.toLowerCase` without brackets.

### A CORRECT ANSWER

```
// Practice 28: a show for each mood, whatever capitals are typed.
// recommend("Happy") should give "Friends".
function recommend(mood) {
  const m = mood.toLowerCase();
  if (m === "happy") {
    return "Friends";
  } else if (m === "curious") {
    return "Dark";
  } else if (m === "brave") {
    return "Avatar: The Last Airbender";
  } else {
    return "The Crown";
  }
}

tryIt(recommend, ["happy"], "Friends");
tryIt(recommend, ["Curious"], "Dark");
tryIt(recommend, ["BRAVE"], "Avatar: The Last Airbender");
tryIt(recommend, ["sleepy"], "The Crown");
```

## Practice 29. Which season is episode 7 in? ~10 min

**Teaches:** Practice: put numbered items into groups with `Math.floor`.

### TASK

Write `seasonOf(episode, perSeason)`. Episodes are numbered from 1 across the whole show, and every season has `perSeason` episodes. Return the season the episode is in.

Fleabag has 6 episodes per season: episode 6 is in season **1**, episode 7 in season **2**.

### COMMON MISTAKES

- `Math.floor(episode / perSeason) + 1` without `- 1`: episode 6 lands in season 2.
- Brackets missing around `episode - 1`, so only the 1 is divided.

### A CORRECT ANSWER

```
// Practice 29: which season is an episode in?
// Fleabag, 6 per season: seasonOf(7, 6) should give 2.
function seasonOf(episode, perSeason) {
  return Math.floor((episode - 1) / perSeason) + 1;
}

tryIt(seasonOf, [1, 6], 1);
tryIt(seasonOf, [6, 6], 1);
tryIt(seasonOf, [7, 6], 2);
tryIt(seasonOf, [12, 6], 2);
```

## Practice 30. Episode number within its season ~10 min

**Teaches:** Practice: the position inside a group with `%`.

### TASK

Write `episodeInSeason(episode, perSeason)`. It returns the episode's number **within its season**.

Fleabag has 6 episodes per season. Episode 8 of the whole show is episode **2** of season 2, and episode 6 is episode **6** of season 1.

### COMMON MISTAKES

- `episode % perSeason`: the last episode of a season gives 0.
- `episode - 1 % perSeason + 1` without brackets: `%` happens first and the answer is the episode itself.

### A CORRECT ANSWER

```
// Practice 30: the episode's number within its own season.
// Fleabag, 6 per season: episodeInSeason(8, 6) should give 2.
function episodeInSeason(episode, perSeason) {
  return (episode - 1) % perSeason + 1;
}

tryIt(episodeInSeason, [8, 6], 2);
tryIt(episodeInSeason, [6, 6], 6);
tryIt(episodeInSeason, [7, 6], 1);
tryIt(episodeInSeason, [1, 6], 1);
```

## Practice 31. Episode 8 as a full label (harder) ~10 min

**Teaches:** Practice: guard against bad input, then build a label from two helpers.

### TASK

Write `fullLabel(episode, perSeason, total)`. It returns:

1. **"No such episode"** when `episode` is less than 1, or more than `total`
2. otherwise a label like **"Season 2, episode 2"**, made with `seasonOf` and `episodeInSeason`

Fleabag has 12 episodes, 6 per season: `fullLabel(8, 6, 12)` gives **"Season 2, episode 2"**.

### COMMON MISTAKES

- Only one guard, so episode 0 becomes "Season 0, episode 0" or episode 13 becomes "Season 3, episode 1".
- `episode >= total`, so the last episode is rejected.
- Writing the helper names without the arguments: `${seasonOf}`.

### A CORRECT ANSWER

```
// Practice 31 (harder): "Season 2, episode 2" from an episode number.

// Helpers, already written (don't change them):
function seasonOf(episode, perSeason) {
  return Math.floor((episode - 1) / perSeason) + 1;
}

function episodeInSeason(episode, perSeason) {
  return (episode - 1) % perSeason + 1;
}

// Your function:
function fullLabel(episode, perSeason, total) {
  if (episode < 1) {
    return "No such episode";
  } else if (episode > total) {
    return "No such episode";
  } else {
    return `Season ${seasonOf(episode, perSeason)}, episode ${episodeInSeason(episode, perSeason)}`;
  }
}

tryIt(fullLabel, [8, 6, 12], "Season 2, episode 2");
tryIt(fullLabel, [6, 6, 12], "Season 1, episode 6");
tryIt(fullLabel, [13, 6, 12], "No such episode");
tryIt(fullLabel, [0, 6, 12], "No such episode");
```

## Practice 32. Which of two shows? (harder) ~10 min

**Teaches:** Practice: a main comparison with a tie-breaker.

### TASK

Write `betterPick(ratingA, runtimeA, ratingB, runtimeB)`. It returns **"First"** or **"Second"**:

1. The show with the **higher rating** wins.
2. With the same rating, the **shorter runtime** wins.
3. When both are the same, return **"Either one"**.

The Wire (9.0, 60 min) against Avatar: The Last Airbender (9.0, 30 min): same rating, so the shorter Avatar wins: **"Second"**.

### COMMON MISTAKES

- Comparing runtimes before ratings, so a short, weaker show wins.
- `runtimeA > runtimeB` for "First": the longer show wins the tie.
- No final `else`, so a perfect tie gives `undefined`.

### A CORRECT ANSWER

```
// Practice 32 (harder): higher rating wins; same rating, shorter wins.
// betterPick(9.0, 60, 9.0, 30) should give "Second".
function betterPick(ratingA, runtimeA, ratingB, runtimeB) {
  if (ratingA > ratingB) {
    return "First";
  } else if (ratingB > ratingA) {
    return "Second";
  } else if (runtimeA < runtimeB) {
    return "First";
  } else if (runtimeB < runtimeA) {
    return "Second";
  } else {
    return "Either one";
  }
}

tryIt(betterPick, [9.2, 60, 8.8, 30], "First");
tryIt(betterPick, [9.0, 60, 9.0, 30], "Second");
tryIt(betterPick, [8.0, 30, 8.0, 60], "First");
tryIt(betterPick, [9.1, 60, 9.1, 60], "Either one");
```

## Practice 33. A summary from three helpers (harder) ~12 min

**Teaches:** Practice: combine three helpers, one call inside another.

### TASK

Write `showSummary(name, year, episodes, runtime)`, using all three helpers. It returns a line like this:

**Chernobyl (2019): 5 episodes, 5 hours**

and for a one-episode, one-hour special: **Pilot (2020): 1 episode, 1 hour**.

### COMMON MISTAKES

- `plural(episodes * runtime / 60, "hour")`: the maths is repeated and not rounded.
- Forgetting the second argument: `plural(hours)` says "118 episodes".
- `${plural(hoursOf, "hour")}`: the helper's name without calling it.

### A CORRECT ANSWER

```
// Practice 33 (harder): one line about a show, from three helpers.
// showSummary("Chernobyl", 2019, 5, 60) should give
// "Chernobyl (2019): 5 episodes, 5 hours"

// Helpers, already written (don't change them):
function titleWithYear(name, year) {
  return `${name} (${year})`;
}

function plural(count, word = "episode") {
  if (count === 1) {
    return `${count} ${word}`;
  } else {
    return `${count} ${word}s`;
  }
}

const hoursOf = (episodes, runtime) => Math.round(episodes * runtime / 60);

// Your function:
function showSummary(name, year, episodes, runtime) {
  return `${titleWithYear(name, year)}: ${plural(episodes, "episode")}, ${plural(hoursOf(episodes,
runtime), "hour")}`;
}

tryIt(showSummary, ["Chernobyl", 2019, 5, 60], "Chernobyl (2019): 5 episodes, 5 hours");
tryIt(showSummary, ["Friends", 1994, 236, 30], "Friends (1994): 236 episodes, 118 hours");
tryIt(showSummary, ["Pilot", 2020, 1, 60], "Pilot (2020): 1 episode, 1 hour");
```

## Practice 34. When does it end? (harder) ~12 min

**Teaches:** Practice: clock times with `Math.floor`, `%` and a leading zero.

### TASK

Write `endTime(hour, minute, runtime)`. It gets the start time and the runtime in minutes, and returns the end time as text, like **"22:15"**. The hours have no extra 0 (**"0:30"** after midnight), but the minutes always have two digits.

Start Friends at 19:00 and watch all 7080 minutes without a break: `endTime(19, 0, 7080)` gives **"17:00"**, some days later.

### COMMON MISTAKES

- No `% 24`, so a late show ends at "24:30".
- Forgetting the leading zero: "23:5".
- `hour + runtime / 60` instead of counting everything in minutes.

### A CORRECT ANSWER

```
// Practice 34 (harder): start at hour:minute, watch for runtime minutes.
// endTime(21, 30, 45) should give "22:15".
function endTime(hour, minute, runtime) {
  const total = hour * 60 + minute + runtime;
  const hours = Math.floor(total / 60) % 24;
  const minutes = total % 60;
  if (minutes < 10) {
    return `${hours}:0${minutes}`;
  } else {
    return `${hours}:${minutes}`;
  }
}

tryIt(endTime, [21, 30, 45], "22:15");
tryIt(endTime, [20, 0, 60], "21:00");
tryIt(endTime, [22, 55, 10], "23:05");
tryIt(endTime, [23, 30, 60], "0:30");
```

## Practice 35. Premiere, finale or regular? (harder) ~12 min

**Teaches:** Practice: five overlapping cases with `%`, strictest first.

### TASK

Write `episodeStatus(episode, perSeason, seasons)`. The show has `perSeason * seasons` episodes in total. Return:

1. **"Not a real episode"** when `episode` is more than the total
2. **"Series finale"** for the very last episode
3. **"Season finale"** for the last episode of a season
4. **"Season premiere"** for the first episode of a season
5. **"Regular episode"** otherwise

### COMMON MISTAKES

- "Season finale" tested before "Series finale", so the last episode is only a season finale.
- `episode / perSeason === 0` instead of `%`.
- Forgetting the total check, so episode 13 is called a premiere.

### A CORRECT ANSWER

```
// Practice 35 (harder): what kind of episode is it?
// Fleabag, 6 per season, 2 seasons: episodeStatus(6, 6, 2) should give "Season finale".
function episodeStatus(episode, perSeason, seasons) {
  const total = perSeason * seasons;
  if (episode > total) {
    return "Not a real episode";
  } else if (episode === total) {
    return "Series finale";
  } else if (episode % perSeason === 0) {
    return "Season finale";
  } else if (episode % perSeason === 1) {
    return "Season premiere";
  } else {
    return "Regular episode";
  }
}

tryIt(episodeStatus, [1, 6, 2], "Season premiere");
tryIt(episodeStatus, [3, 6, 2], "Regular episode");
tryIt(episodeStatus, [6, 6, 2], "Season finale");
tryIt(episodeStatus, [12, 6, 2], "Series finale");
tryIt(episodeStatus, [13, 6, 2], "Not a real episode");
```

## Practice 36. How much longer? (harder) ~12 min

**Teaches:** Practice: a sentence that changes with a comparison, always with positive numbers.

### TASK

Write `compareTime(nameA, hoursA, nameB, hoursB)`. It returns one of three sentences:

- **"Friends takes 56 hours more than Breaking Bad"** (118 against 62)
- **"Chernobyl takes 5 hours less than Band of Brothers"** (5 against 10)
- **"The Wire and The Crown take the same time"** (60 and 60)

The numbers in the sentences are always positive.

### COMMON MISTAKES

- `${hoursA - hoursB}` in every case, so the sentence says "-5 hours less".
- The words "more" and "less" in the wrong blocks.
- No case for equal times, so it says "0 hours less".

### A CORRECT ANSWER

```
// Practice 36 (harder): compare how long two shows take, in a sentence.
// compareTime("Friends", 118, "Breaking Bad", 62) should give
// "Friends takes 56 hours more than Breaking Bad"
function compareTime(nameA, hoursA, nameB, hoursB) {
  if (hoursA > hoursB) {
    return `${nameA} takes ${hoursA - hoursB} hours more than ${nameB}`;
  } else if (hoursA < hoursB) {
    return `${nameA} takes ${hoursB - hoursA} hours less than ${nameB}`;
  } else {
    return `${nameA} and ${nameB} take the same time`;
  }
}

tryIt(compareTime, ["Friends", 118, "Breaking Bad", 62], "Friends takes 56 hours more than Breaking
Bad");
tryIt(compareTime, ["Chernobyl", 5, "Band of Brothers", 10], "Chernobyl takes 5 hours less than Band
of Brothers");
tryIt(compareTime, ["The Wire", 60, "The Crown", 60], "The Wire and The Crown take the same time");
```

## Extra 1. Episode codes like S01E05 ~15 min

**Teaches:** `String(n).padStart(2, "0")` gives two-digit numbers like 05.

### TASK

Write `episodeCode(season, episode)` that returns a code like **S01E05**. Each card gets a new line: *First episode: S01E01*.

Test it: `console.log(episodeCode(10, 18));` should print **S10E18**.

### A CORRECT ANSWER

```
// ... unchanged ...

console.log(plural(3, "season"));

function episodeCode(season, episode) {
  const s = String(season).padStart(2, "0");
  const e = String(episode).padStart(2, "0");
  return `S${s}E${e}`;
}

console.log(episodeCode(10, 18));
```

## Extra 2. How many days to binge it? ~15 min

**Teaches:** `Math.ceil()` rounds up to the next whole number.

### TASK

Write `binge(hours, hoursPerDay)` that returns how many days it takes to watch `hours` of TV at `hoursPerDay` a day. A started day counts as a whole day, so round up.

Every card gets a new line: *At 2 hours a day: 59 days*.

### A CORRECT ANSWER

```
// ... unchanged ...

console.log(plural(3, "season"));

function binge(hours, hoursPerDay) {
  return Math.ceil(hours / hoursPerDay);
}
```